

SH-Analyzer — Contexto do Projeto

Documento de referência compacto para uso como contexto por assistentes de IA • Atualizado em 28/08/2026 • Substitui "Contexto do Projeto-1.pdf"

Propósito deste arquivo: os arquivos reais do projeto (firmware em C/Assembly, script de análise em Python, arquivos de hardware) são grandes demais para serem enviados de uma vez a um assistente de IA. Este documento resume a arquitetura, o estado atual, as decisões técnicas já validadas e os próximos passos do projeto num único lugar compacto, para dar contexto suficiente sem precisar reenviar o código-fonte inteiro a cada conversa.

1. Visão Geral do Projeto

SH-Analyzer (Analisador de Supraharmônicos) é um projeto de hardware e firmware embarcado de alto desempenho para identificar **supraharmônicos** — perturbações de alta frequência (dezenas de kHz) introduzidas por conversores eletrônicos de potência em tensão/corrente de redes elétricas — que normalmente escapam de analisadores de Qualidade de Energia Elétrica (QEE) convencionais.

O sistema é formado por (a) um **frontend analógico** (PCB própria, com isolamento galvânico) que condiciona os sinais de tensão/corrente da rede, e (b) um **microcomputador industrial BeagleBone** que faz a aquisição de altíssima frequência via uma arquitetura híbrida **PRU (tempo real) + ARM (Linux)**. Os dados brutos são gravados em disco e depois pós-processados (FFT, filtragem, conversão de formato) por um script Python dedicado (seção 6).

Contexto acadêmico

- Iniciação Científica (IC) — "Sistema de identificação da presença de supraharmônicos em redes e cargas elétricas", curso de Engenharia de Controle e Automação, Unicamp.
- Orientação: Prof. Dr. José Antenor Pomilio. Coorientação: Dr. Mateus Pinheiro Dias.
- Repositório sob licença MIT; design da PCB e firmware por Gabriel Canela.

2. Hardware

2.1 Plataforma de aquisição

BeagleBone: processador **ARM Cortex-A8 (Linux)** + subsistema **PRU-ICSS** (Programmable Real-Time Unit). A PRU0 faz o controle determinístico via bit-banging SPI e grava as amostras diretamente numa região reservada da DDR; o ARM só extrai blocos prontos e grava em disco, sem processamento em tempo real.

2.2 Componentes principais (nomes exatos, para referência)

Componente	Peça	Função no projeto	Especificações-chave
ADC	Texas Instruments ADS8688 (datasheet SBAS582C)	Conversão A/D dos canais de tensão/corrente do frontend analógico.	8 canais single-ended endereçáveis (0–7); 16 bits; throughput agregado máximo de 500 kSPS somando todos os canais (teto físico do chip — com N canais intercalados, o limite cai para 500 kSPS/N); atualmente operado em modo manual (comando de seleção de canal enviado a cada quadro SPI); possui também um modo automático (Auto/Auto_RST) ainda não implementado neste projeto (ver seção 8, próximo passo #1).

Componente	Peça	Função no projeto	Especificações-chave
Isolador SPI	Analog Devices ADuM3150 (a confirmar contra o esquemático/BOM — usuário não tem 100% de certeza do part number exato)	Isolamento galvânico entre a PRU/lado digital e o frontend analógico conectado diretamente à rede elétrica.	Família SPI Isolator, baseada em iCoupler (transformador integrado — isolamento <u>magnético</u> , não optoacoplador); 6 canais dedicados (4 de alta velocidade para CLK/MOSI/MISO/CS + 2 auxiliares de baixa velocidade); isolamento de 3,75 kV; suporta até 40 MHz em modo delay-clock ou 17 MHz em modo 4 fios.

ATENÇÃO: Reconciliação a verificar: o README e os comentários de depuração do projeto atribuem o bug de saturação SPI em aberto (seção 7.2) à hipótese de "assimetria de tempo de subida/descida num optoacoplador". Se o isolador realmente instalado for o ADuM3150 (isolador digital por transformador, com propagation delay/skew especificados no datasheet, não um optoacoplador simples tipo PC817), essa hipótese específica de causa pode precisar ser revisada — mas o método de diagnóstico já planejado (comparar o sinal antes/depois do isolador, eliminar jumpers longos) continua válido independentemente da causa raiz exata.

2.3 PCB

Projeto "DAQ_Module" desenvolvido integralmente no **Altium Designer** (hardware/DAQ_Module/): esquemático, layout, visualização 3D, BOM e arquivos Gerber. Inclui isolamento galvânico entre a PRU e o frontend conectado à rede.

3. Firmware — Arquitetura Híbrida PRU + ARM

3.1 Divisão de responsabilidades

- **PRU0 (spi_core.asm, Assembly):** laço crítico de tempo — bit-banging SPI com o ADS8688 e gravação direta das amostras num par de buffers "ping-pong" na DDR.
- **pru_main.c:** inicialização mínima da PRU antes de chamar a rotina em Assembly (resource_table exigida pelo remoteproc do Linux, clamps de segurança para sample_period_ticks e num_canais caso a RAM compartilhada esteja "fria").
- **ARM/Linux (ler_adc.c, C):** mapeia a região de controle e os buffers via /dev/mem, configura canais/frequência, e despeja os blocos prontos em disco (.bin) o mais rápido possível.

3.2 Protocolo SPI com o ADS8688 (modo manual, em uso hoje)

- Frame de **32 ciclos de SCLK por amostra**: 16 ciclos para escrever o comando de seleção de canal (macro CMD_BIT) + 16 ciclos para ler o dado da conversão **anterior** (macro DATA_BIT) — assim funciona o modo manual do ADS8688.
- Pinagem na PRU: bit 0 de r30 = SCLK, bit 1 = MOSI/SDI, bit 2 de r31 = MISO/SDO, bit 3 de r30 = CS.
- MISO amostrado no último instante seguro antes da borda de descida do SCLK (chave para resolver o bug histórico de leitura presa em fundo de escala).
- Atrasos de CS calibrados empiricamente (ainda não comparados formalmente ao datasheet): setup ≈1 us (200 ciclos), hold ≈0,5 us (100 ciclos), tempo mínimo CS-alto entre transações ≈0,5 us (100 ciclos).
- Comando de canal (32 bits, alinhado ao bit 31): 0xC0000000 | (canal << 26) — canal 0 → 0xC0000000, canal 1 → 0xC4000000 (era o valor fixo "MAN_Ch_1" antes do suporte multi-canal), ... até canal 7.

3.3 Estrutura de sincronização ARM ↔ PRU — shared_control (memoria_pru.h)

Struct mapeada na RAM compartilhada da PRU-ICSS (12 KB, endereço físico 0x4A310000 / endereço PRU 0x00010000). Handshake via config_ready: a PRU só começa a gravar depois que o ARM já escreveu toda a configuração.

Offset	Campo	Escrito por	Descrição
0	sample_period_ticks	ARM (relido a cada iteração pela PRU)	Período entre amostras em ciclos de PRU (200 MHz).

Offset	Campo	Escrito por	Descrição
4	active_buffer	PRU	Qual buffer (0/1) está ativo no momento.
8 / 12	buffer_0_ready / buffer_1_ready	PRU seta, ARM zera	Sinaliza que um buffer está pronto para leitura.
16 / 20	buffer_0_addr / buffer_1_addr	ARM	Endereços físicos na DDR reservada.
24	config_ready	ARM seta 1; PRU zera após ler	Handshake inicial de configuração.
28	num_canais	ARM (lido 1x pela PRU)	Nº de canais ativos na captura (1–8).
32–63	comandos_canais[8]	ARM (lido 1x pela PRU)	Tabela de comandos de 32 bits já prontos para r28, um por canal ativo, na ordem de captura.

3.4 Buffers ping-pong e memória

- 16 MB reservados na DDR (0x9F000000) fora do alcance do gerenciador de memória do Linux; hoje só **4 MB são usados** (2 buffers × 2 MB = 2 × 1.048.576 amostras × 2 bytes).
- SAMPLES_PER_BUFFER = **1.048.576** — precisa bater manualmente com o "LDI r21" hardcoded em spi_core.asm (não há #define no Assembly).
- **Risco documentado, ainda não mitigado:** a troca de buffer na PRU não checa se o ARM já terminou de processar o buffer anterior (sem backpressure) — corrupção silenciosa é possível se o ARM atrasar. Risco cresce justamente ao subir a frequência de amostragem (ver seção 9 e seção 8, passo 3).

3.5 Build e deploy

- firmware/Makefile: make compila o ARM (gcc -O3 → ler_adc) e a PRU (clpru --silicon_version=3 → fw_pru.out, a partir de pru_main.c + spi_core.asm + AM335x_PRU.cmd).
- firmware/setup.sh: configura os 4 pinos via config-pin e recarrega fw_pru.out no remoteproc (stop/start). Contém aviso próprio sobre o nome do binário: copiar um nome antigo/errado não gera erro visível no terminal, só um firmware desatualizado rodando silenciosamente.

4. Captura Multi-Canal

ATENÇÃO: ATUALIZAÇÃO DE STATUS: o README ainda descreve a parte em Assembly deste recurso como "aguardando validação em hardware". Isso está **SUPERADO** — a leitura multi-canal em Assembly (spi_core.asm) já foi implementada e **testada com sucesso em hardware real**.

- Vários canais do ADS8688 são intercalados (round-robin) num único par de buffers, sem buffers separados por canal e sem cabeçalho no arquivo.
- Índice de canal (r6, na PRU) percorre ctrl->comandos_canais[] e **nunca é reiniciado na troca de buffer** — só ao completar num_canais — o que mantém a intercalação em fase durante toda a captura, mesmo com SAMPLES_PER_BUFFER não sendo múltiplo do número de canais.
- **Atraso de pipeline de 1 quadro do ADS8688:** em modo manual, cada quadro devolve o resultado do comando enviado no quadro *anterior*. Com 1 canal isso é invisível; com vários, desalinhará (posição no arquivo) ↔ (canal) em 1 posição. Corrigido deliberadamente do lado ARM (não na PRU, para não pressionar mais os 8 KB de PRU_IMEM): ler_adc.c descarta a primeiríssima amostra bruta de toda a captura.
- Uso: sudo ./ler_adc <freq_hz> [lista_canais] — lista opcional, ex. "0,1,3" (sem espaços); sem ela, captura só o canal 1 (padrão histórico). Validação: 0–7, sem repetição, no máx. 8 canais.
- Formato do .bin resultante: amostra bruta na posição *i* pertence a canais[i % num_canais]. O .bin não carrega esse metadado — o adc_tool.py precisa receber a mesma lista/ordem via --canais para desintercalar corretamente.

5. scripts/Adc_tool.py — Ferramenta de Análise (recém-adicionada, versão completa)

Renomeado do antigo `plot_adc.py` — deixou de fazer só plotagem. Declara dependências inline (PEP 723: `matplotlib, numpy, scipy, PyQt6`), então roda via `uv run scripts/adc_tool.py ...` sem instalação manual.

5.1 Dois modos de operação

- **Conversão** (`-c/--converter + -o/--saida`): converte entre `.bin` (binário bruto, 2 bytes/amostra, gerado por `ler_adc.c`) e `.csv` (texto legível), nos dois sentidos, detectando a direção pelas extensões. Processa em blocos (streaming) — não carrega arquivos grandes inteiros na memória.
- **Plotagem** (padrão): plota forma de onda no tempo e, opcionalmente (`--fft`), o espectro de frequência. Aceita `.bin` (via `numpy.memmap`, leve) ou `.csv` (carregado inteiro).

5.2 FFT sem vazamento espectral (spectral leakage)

Estratégia de 5 passos: (1) estimativa grosseira da fundamental via FFT+Hann numa faixa configurável (`--freq-min/--freq-max`, padrão 45–65 Hz); (2) filtro passa-baixa Butterworth fixo em ordem 4 para isolar a fundamental; (3) cruzamentos por zero ascendentes **interpolados linearmente** (posição fracionária, não presa à grade de amostragem); (4) refinamento do período usando TODOS os ciclos disponíveis (último–primeiro)/`n_ciclos`, diluindo o erro; (5) corte do trecho exatamente em ciclos inteiros antes da FFT principal, que ataca a causa raiz do vazamento espectral.

5.3 Recursos principais

- **Janelas espectrais** (`--janela`): retangular/boxcar (padrão — o recorte em ciclos inteiros já faz o trabalho pesado), hann, blackman-harris (~92 dB de lóbulo lateral), flattop (melhor exatidão de amplitude), kaiser (+ `--kaiser-beta` ajustável). Correção por ganho coerente sempre aplicada.
- **--fft N**: analisa só os primeiros N ciclos completos, para investigar distúrbios momentâneos — combinável com `--inicio/--fim` para posicionar a janela.
- **Multi-canal completo**: `--canais` (lista/ordem da captura), `--canais-exibir` (subconjunto), `--layout-canais` {separados, sobrepostos}; FFT independente por canal; calibração `--faixa/--ganho/--offset` aceitam valor único OU uma lista (1 por canal, ex. canal de tensão + canal de corrente com ganhos diferentes).
- **Filtragem digital opcional** (`--filtro-passa-baixa/--filtro-passa-alta`, `--ordem-filtro` 4–8, padrão 5): Butterworth em SOS + `sosfiltfilt` (fase zero, evita deslocar os cruzamentos por zero), aplicada ANTES de qualquer outra etapa (recorte de ciclos, FFT, plotagem no tempo). Não afeta a conversão `.bin↔.csv`.
- Conversão `.bin→.csv` ganha coluna "canal" quando há mais de 1 canal (CSV fica autodescritivo: `.csv→.bin` não precisa mais de `--canais` nesse caso).
- Validado ponta a ponta com dados sintéticos multi-canal, incluindo round-trip `.bin → .csv → .bin` sem perdas.

6. Status Atual

6.1 Já validado em hardware

- Protocolo de 32 ciclos do ADS8688 em modo manual (16 comando + 16 dado).
- Handshake `config_ready` entre ARM e PRU.
- Ressincronização periódica do registrador CYCLE da PRU (trava ao estourar 32 bits em vez de dar a volta, ~21,47 s a 200 MHz — sem isso capturas longas travavam sozinhas).
- Inicialização explícita de CS/SCLK/MOSI em repouso antes do laço principal.
- Uso de laços de atraso (em vez de NOP repetido) para controlar a velocidade do SPI sem estourar os 8 KB de `PRU_IMEM`.
- **Captura multi-canal completa**, incluindo o índice round-robin em Assembly (atualização — ver seção 4).

6.2 Em aberto — bug de saturação SPI

A comunicação SPI ainda satura no valor de fundo de escala (leitura constante, independente da tensão real de entrada), mesmo em velocidades bem mais lentas que o firmware original comprovadamente funcional (`backup-pre-assembly/teste_spi_pru.c`). Diagnóstico do "preâmbulo" (16 bits que deveriam ser sempre zero) indica um padrão de transição único e consistente, característico de assimetria de tempo de subida/descida num isolador (ver

nota de reconciliação na seção 2.2 sobre a identidade exata do componente). Próximo passo já cogitado: eliminar jumpers longos e conectar as placas diretamente, para isolar se a causa é integridade de sinal.

7. Próximos Passos (plano atual do usuário)

Passo 1 — Modo automático do ADS8688

Implementar o modo automático (Auto/Auto_RST) do ADS8688. Hoje o firmware envia um comando de seleção de canal a **cada amostra** (mesmo em captura de 1 canal só). O modo automático elimina esse reenvio: o host programa a sequência de canais uma única vez e o próprio ADC avança sozinho a cada frame. Isso é uma mudança de **protocolo** (mais arriscada que ajuste de timing) — deve ser tratada como experimento isolado, validado em hardware com o mesmo rigor já aplicado ao modo manual, preservando retrocompatibilidade byte-a-byte do formato de captura de 1 canal.

Passo 2 — Validação em laboratório controlado (a 102,4 kHz)

Ida ao laboratório para gerar uma **rede elétrica limpa e controlada** e injetar ruído em **frequências específicas conhecidas**, medindo com o SH-Analyzer. Comparar o espectro medido contra o ruído efetivamente injetado demonstra que os dados coletados **não estão sendo corrompidos** — o objetivo é dar credibilidade científica às leituras do sistema. Toda essa etapa será feita a **102,4 kHz**, frequência que já está empiricamente garantida como estável/funcional hoje (não confundir com a estimativa teórica mais conservadora de ~48 kHz mencionada em docs/melhorias-propostas.md item 1.1 — essa estimativa é especulativa e ainda não foi confirmada por instrumentação real, ver seção 9).

Passo 3 — Aumento gradual da frequência (se sobrar tempo)

Partindo de 102,4 kHz (validado), aumentar a frequência de amostragem aos poucos até os dados começarem a falhar/corromper, para mapear o teto real do sistema completo (não só do ADC). Atenção: é justamente na direção de frequências mais altas que o risco de corrupção silenciosa do esquema ping-pong sem backpressure (seção 3.4 / seção 9) fica mais provável — um resultado "ruim" nesse teste pode ser corrupção de buffer, não necessariamente limite físico do ADC/isolador/PRU.

Passo 4 — Diagnóstico de gargalo

Ao identificar um gargalo/limite, isolar em qual elo da cadeia ele está: **ADS8688** (teto físico de 500 kSPS agregado), **ADuM3150 / isolador SPI** (limite de banda ou distorção introduzida pelo isolamento), **PRU** (timing do bit-banging, folga de PRU_IMEM) ou o **firmware** em si (implementação atual do laço SPI, atrasos fixos ainda não comparados ao datasheet — ver seção 9).

Passo 5 — Relatório final

Com todos os dados de laboratório coletados: gerar as imagens/gráficos (via `adc_tool.py` — forma de onda, espectros, comparações canal a canal) e escrever o relatório final da Iniciação Científica.

8. Notas de Desempenho / Otimização (referência: docs/melhorias-propostas.md)

Documento de revisão técnica completo já existe no repositório (`docs/melhorias-propostas.md`) cobrindo taxa de amostragem (firmware+hardware) e reorganização de projeto/portfólio. **Não** cobre o `adc_tool.py` (deixado de fora por pedido explícito). Pontos mais relevantes para o plano da seção 7:

- **Gargalo provável hoje é a implementação SPI da PRU, não o ADC/barramento:** um comentário histórico em `ler_adc.c` estimava ~21 us/transação (~47–48 kHz real) contra os 500 kSPS de teto do ADS8688 — mas essa é uma **estimativa especulativa antiga, ainda não confirmada por instrumentação**, e parece inconsistente com 102,4 kHz já funcionando de forma estável hoje. Antes de otimizar, o documento recomenda instrumentar com um pino de PRU "sonda" para medir onde o tempo por transação realmente vai.
- **Risco de corrupção silenciosa no ping-pong sem backpressure** (já citado na seção 3.4) — relevante diretamente para o passo 3 da seção 7.

- Lado ARM sem tocar Assembly: checar retorno de `fwrite()`, `BLOCOS_PARA_CAPTURAR` hoje fixo em 1 (só 1 buffer por captura — o ping-pong quase não é exercitado em regime contínuo), `SCHED_FIFO` + `mlockall()` para reduzir jitter de escalonamento, revisar o `usleep(2000)` fixo do polling.
- `SAMPLES_PER_BUFFER` e o layout de comandos `canais[]` são sincronizados **manualmente** entre `memoria_pru.h` (C) e `spi_core.asm` (hardcoded em hexadecimal) — risco documentado, sem checagem automática ainda.
- Hipótese de modo automático do ADS8688 (Auto/Auto_RST) e ampliação do uso da DDR reservada (hoje só 4 MB dos 16 MB) também estão catalogadas como melhorias de médio/alto impacto no documento de referência.

9. Estrutura do Repositório (inventário de arquivos)

Caminho	Conteúdo / papel
README.md	Visão geral, arquitetura, status de depuração, guia de uso completo.
LICENSE	MIT.
.gitignore	Artefatos de build, dados coletados (.bin/.csv), ambiente Python/editor.
firmware/setup.sh	Deploy: config-pin dos 4 pinos + carrega <code>fw_pru.out</code> no remoteproc.
firmware/Makefile	<code>make</code> → compila ARM (<code>ler_adc</code> , <code>gcc</code>) e PRU (<code>fw_pru.out</code> , <code>clpru</code>).
firmware/AM335x_PRU.cmd	Linker script da PRU: mapa de memória e seções.
firmware/memoria_pru.h	Layout de <code>shared_control</code> + constantes compartilhadas C/Assembly (seção 3.3).
firmware/pru_main.c	Inicialização mínima da PRU + <code>resource_table</code> p/ remoteproc.
firmware/spi_core.asm	Laço crítico bit-banging SPI (32 ciclos/amostra), round-robin multi-canal — TESTADO em hardware (seção 4).
firmware/ler_adc.c	Programa ARM: parsing de args, config de <code>shared_control</code> , grava .bin, descarte de amostra de alinhamento multi-canal.
scripts/Adc_tool.py	Conversão .bin↔.csv + plotagem + FFT + filtros digitais, multi-canal completo — RECÉM-ADICIONADO (seção 5).
hardware/DAQ_Module/	Projeto Altium Designer (esquemático + PCB) do frontend analógico/DAQ.
docs/melhorias-propostas.md	Revisão técnica: taxa de amostragem + reorganização/profissionalização do repo (seção 8).
docs/Contexto do Projeto-2.pdf	Este documento (versão anterior: -1.pdf).
backup pre-assembly/	Protótipo funcional em C puro, pré-reescrita em Assembly — referência de comportamento correto (<code>teste_spi_pru.c</code>).

Documento gerado para uso como contexto de IA — mantenha-o atualizado sempre que houver mudanças estruturais relevantes no projeto (novos componentes, mudanças de protocolo, resultados de laboratório, etc.).